ECE 661: Homework #4
Pruning and Fixed-point Quantization

Yiran Chen

ECE Department, Duke University — Fall 2026

Objectives

Homework #4 covers the contents of Lectures 14 $\sim$ 18. This assignment starts with conceptual questions on model pruning and quantization techniques, followed by lab questions evaluating the effectiveness of different sparse optimization methods on linear models, iterative pruning of a pretrained CNN model, and training fixed-point quantized CNNs.

Warning: You are asked to complete the assignment independently.

This assignment has **100** required points and **10** optional bonus points. The final score cannot exceed 100 points, following the current homework policy. You must submit your report in PDF format and your original codes for the lab questions through **Gradescope** by the deadline announced on Canvas. You need to submit **three individual files** including:

1. A **self-contained** PDF report, which provides (1) answers and explanations for all T/F questions and (2) **relevant code snippets**, results, and explanations for all lab tasks.
2. *A single notebook file* used to produce the results for Lab1.
3. the completed notebook file `hw4.ipynb`, respectively for Lab 2 and Lab 3.

The bonus has a separate notebook and automatic evaluation workflow, described at the end of this handout. No additional written report is required for the bonus. The three files above are for the required questions.

Note: A 20% deduction per question will be applied if the self-contained PDF omits code snippets, includes raw (unformatted or unsummarized) outputs, or fails to follow the required format. Additionally, do not submit a PDF generated directly from your Jupyter Notebook, as this increases the TA's grading workload. Manually written reports are also highly discouraged.

1 True/False Questions (15 pts)

For each question, please provide a short explanation to support your judgment.

Problem 1.1 (3 pts) Using sparsity-inducing regularizers like L-1 in DNN optimization with SGD guarantees exact zero values in weight elements, making further pruning unnecessary.

Problem 1.2 (3 pts) Iterative pruning with projected gradient descent (PGD) will lead to hardware-friendly, structured model sparsity.

Problem 1.3 (3 pts) Weight sharing in Deep Compression reduces both storage memory and computation cost equally.

Problem 1.4 (3 pts) Compared to the standard Lasso regularizer, Group Lasso can lead to unstructured sparsity of DNNs, which is less hardware-friendly.

Problem 1.5 (3 pts) In Trained Ternary Quantization (TTQ), only the ternary weights are learned during training, while the scaling coefficients for positive and negative weights are fixed.

2 Lab 1: Sparse optimization of linear models (30 pts)

By now you have seen multiple ways to induce a sparse solution in the optimization process. This problem will provide you some examples under linear regression setting so that you can compare the effectiveness of different methods. For this problem, consider the case where we are trying to find a sparse weight W that can minimize $L = \sum_i (X_i W - y_i)^2$. Specifically, we have $X_i \in \mathbb{R}^{1\times 5}$, $W \in \mathbb{R}^{5\times 1}$ and $||W||_0 \leq 2$.

For Problem (a) - (f), consider the case where we have 3 data points: $(X_1 = [-2, 2, 1, 0, -1], y_1 = 3)$; $(X_2 = [-2, 2, -2, 1, 1], y_2 = 1)$; $(X_3 = [1, 0, -2, 1, -1], y_3 = 1)$. For stability the objective L should be minimized through full-batch gradient descent, with initial weight W^0 set to $[0; 0; 0; 0; 0]$ and use learning rate $\mu = 0.02$ throughout the process. Please run gradient descent for 200 steps for all the following problems. **For $log(L)$ plot, please use matplotlib.pyplot.yscale('log')**

Lab 1 (30 points)

(a) (4 pts) Theoretical analysis: with learning rate μ, suppose the weight you have after step k is W^k, derive the symbolic formulation of weight W^{k+1} after step k+1 of full-batch gradient descent with $X_i, y_i, i \in \{1, 2, 3\}$. (Hint: note the loss L we have is defined differently from standard MSE loss.)

(b) (3 pts) In Python, directly minimize the objective L without any sparsity-inducing regularization/constraint. Plot the value of $log(L)$ vs. #steps throughout the training, and use another figure to plot how the value of each element in W is changing throughout the training. From your result, is W converging to an optimal solution? Is W converging to a sparse solution?

(c) (6 pts) Since we have the knowledge that the ground-truth weight should have $||W||_0 \leq 2$, we can apply **projected gradient descent** to enforce this sparse constraint. Redo the optimization process in (b), this time prune the elements in W after every gradient descent step to ensure $||W^l||_0 \leq 2$. Plot the value of $log(L)$ throughout the training, and use another figure to plot the value of each element in W in each step. From your result, is W converging to an optimal solution? Is W converging to a sparse solution?

(d) (5 pts) In this problem we apply ℓ_1 regularization to induce the sparse solution. The minimization objective therefore changes to $L + \lambda||W||_1$. Please use full-batch gradient descent to minimize this objective, with $\lambda = \{0.2, 0.5, 1.0, 2.0\}$ respectively. For each case, plot the value of $log(L)$ throughout the training, and use another figure to plot the value of each element in W in each step. From your result, comment on the convergence performance under different λ.

(e) (6 pts) Here we optimize the same objective as in (d), this time using **proximal gradient update**. Recall that the proximal operator of the ℓ_1 regularizer is the soft thresholding function. Set the threshold in the soft thresholding function to $\{0.004, 0.01, 0.02, 0.04\}$ respectively. Plot the value of $log(L)$ throughout the training, and use another figure to plot the value of each element in W in each step. Compare the convergence performance with the results in (d). (Hint: Optimizing $L + \lambda||W||_1$ using gradient descent with learning rate μ should correspond to proximal gradient update with threshold $\mu\lambda$)

(f) (6 pts) Trimmed ℓ_1 ($T\ell_1$) regularizer is proposed to solve the "bias" problem of ℓ_1. For simplicity you may implement the $T\ell_1$ regularizer as applying a ℓ_1 regularization with strength λ on the 3 elements of W **with the smallest absolute value**, with no penalty on other elements. Minimize $L + \lambda T\ell_1(W)$ **using proximal gradient update** with $\lambda = \{1.0, 2.0, 5.0, 10.0\}$ (correspond the soft thresholding threshold $\{0.02, 0.04, 0.1, 0.2\}$). Plot the value of $log(L)$ throughout the training, and use another figure to plot the value of each element in W in each step. Comment on the convergence comparison of the Trimmed ℓ_1 and the ℓ_1. Also compare the behavior of the early steps (e.g. first 20) between the Trimmed ℓ_1 and the iterative pruning.

3 Lab 2: Pruning ResNet-20 model (25 pts)

ResNet-20 is a popular convolutional neural network (CNN) architecture for image classification. Compared to early CNN designs such as VGG-16, ResNet-20 is much more compact. Thus, conducing the model compression on ResNet-20 is more challenging.

This lab explores the element-wise pruning of ResNet-20 model on CIFAR-10 dataset. We will observe the difference between single step pruning and iterative pruning, plus exploring different ways of setting pruning threshold. Everything you need for this lab can be found in `HW4.zip`.

Lab 2 (25 points)

(a) (2 pts) In `hw4.ipynb`, run through the first three code blocks, report the accuracy of the floating-point pretrained model.

(b) (6 pts) Complete the implementation of *pruning by percentage* function in the notebook. Here we determines the pruning threshold in each DNN layer by the **'q-th percentile'** value in the absolute value of layer's weight element. Use the next block to call your implemented *pruning by percentage*. Try pruning percentage q = 10, 30, 50, 70, 90 (in percent, as the `q` argument of the function). Report the test accuracy q. (**Hint:** You need to reload the full model checkpoint before applying the prune function with a different q).

(c) (6 pts) Fill in the `finetune_after_prune` function for pruned model finetuning. Make sure the pruned away elements in previous step are kept as 0 throughout the finetuning process. Finetune the pruned model with q = 85 for 20 epochs with the provided training pipeline. Report the best accuracy achieved during finetuning. Finish the code for sparsity evaluation to check if the finetuned model preserves the sparsity.

(d) (5 pts) Implement iterative pruning. Instead of applying single step pruning before finetuning, try iteratively increasing the sparsity of the model before each epoch of finetuning. Linearly increase the pruning percentage for 10 epochs until reaching 85% in the final epoch (prune $(8.5 \times e)\%$ before epoch e) then continue finetune for 10 epochs. Pruned weights can be recovered during the iterative pruning process before the final pruning step. Compare the performance with (c).

(e) (6 pts) Perform magnitude-based global iterative pruning. Previously we set the pruning threshold of each layer following the weight distribution of the layer and prune all layers to the same sparsity. This will constrain the flexibility in the final sparsity pattern across layers. In this question, Fill in the `global_prune_by_percentage` function to perform a global ranking of the weight magnitude from all the layers, and determine a single pruning threshold by percentage for all the layers. Repeat iterative pruning to 85% sparsity, and report final accuracy and the percentage of zeros in each layer.

4 Lab 3: Fixed-point quantization and finetuning (30 pts)

Besides pruning, fixed-point quantization is another important technique applied for deep neural network compression. In this Lab, you will convert the ResNet-20 model we used in previous lab into a quantized model, evaluate is performance and apply finetuning on the model.

Lab 3 (30 points)

(a) (15 pts) As is mentioned in lecture 18, to train a quantized model we need to use floating-point weight as trainable variable while use a straight-through estimator (STE) in forward and backward pass to convert the weight into quantized value. Intuitively, the forward pass of STE converts a float weight into fixed-point, while the backward pass passes the gradient straightly through the quantizer to the float weight.

To start with, implement the STE forward function in `FP_layers.py`, so that it serves as a linear quantizer with dynamic scaling, as introduced on page 9 of lecture 18. Please follow the comments in the code to figure out the expected functionality of each line. **Copy the code to your PDF report, format it so that it is readable and add comments** in the finished STE class. Submission of the `FP_layers.py` file is not required. (**Hint:** Please consider zeros in the weight as being pruned away, and build a mask to ensure that STE is only applied on non-zero weight elements for quantization.)

(b) (5 pts) In `hw4.ipynb`, load pretrained ResNet-20 model, report the accuracy of the floating-point pretrained model. Then set `Nbits` in the first line of the first code block under "Lab 3 (b) and (c)" to 5, 4, 3, and 2 respectively, run it and report the test accuracy you got. (Hint: In this block the line defining the ResNet model (second line) will set the residual blocks in all three stages to `Nbits` fixed-point, while keeping the first conv and final FC layer still as floating point.)

(c) (5 pts) With `Nbits` set to 5, 4, 3, and 2 respectively, run the provided code to finetune the quantized model for 20 epochs. You do not need to change other parameter in the `finetune` function. For each precision, report the highest testing accuracy you get during finetuning. Comment on the relationship between precision and accuracy, and on the effectiveness of finetuning.

(d) (5 pts) In practice, we want to apply both pruning and quantization on the DNN model. Here we explore how pruning will affect quantization performance. Please load the checkpoint of the 85% sparsity model with the best accuracy from Lab 2, repeat the process in (c), report the accuracy before and after finetuning, and discuss your observations comparing to (c)'s results. The provided finetuning code for this part keeps the pruned weights at zero (`keep_sparsity=True`), so the model stays 85% sparse.

Bonus: A precision budget for a decision model (10 pts)

TypeSafe's Jev, announced in September 2026, has renewed interest in "System One" models, which return structured decisions and associated probabilities for tasks such as routing customer requests.[1] Together AI has since published a tutorial on building a Jev-style classifier from an open model.[2] Such systems make a large number of small decisions, so the cost of storing and serving the model matters. This bonus asks how far such a model can be compressed before its decisions change.

The supplied open-weight model, Decision-1.0-Kai, uses a ModernBERT encoder to score a request against a list of candidate options and returns one probability per option. Jev motivates the exercise, but all experiments use Kai. The work is provided as the notebook `HW4-Bonus.ipynb`, which runs in Google Colab or in the environment used for Labs 2 and 3. A GPU runtime is recommended, and a CPU also works. The setup cell downloads the course support code and the model. A provided function, `hw.decide`, shows how the original and compressed models distribute probability across options for requests you write.

Lab 4 (Bonus 10 points)

(a) (4 pts) Implement symmetric per-row quantization for a two-dimensional `torch.float32` weight matrix W. For each row i, compute the scale $s_i = \max_j |W_{ij}|/127$, then divide the row by s_i, round to the nearest integer, clip to the range $[-127, 127]$, and convert the result to `torch.int8`.

Return the pair `(quantized, scale)`. The first element must have the same shape as W, and the second must be a `torch.float32` tensor of shape `(rows, 1)`. The function must not modify its input. An all-zero row must map to zeros and still receive a positive scale.

(b) (6 pts) Choose a precision budget. Assign a bit width (2, 3, 4, 6 or 8) and a clipping fraction to each of three weight groups: the token embeddings, the attention layers and the feed-forward layers. The notebook applies your choices and reports the model size and how many decisions changed.

Your policy earns points if the compressed model makes the same decision as the original model on at least 95% of the held-out requests. It then earns 3 points at 319 MB or less, or 6 points at 270 MB or less. For reference, the int8 model from (a) is 368 MB.

Submission and grading. Upload the completed notebook file to the Gradescope assignment "ECE661 Homework 4 - Bonus (notebook)". Only the two tagged cells are graded, the function for (a) and the policy for (b). Part (a) is graded within minutes of each submission. For (b), Gradescope reports the exact size of the policy immediately, and the points are posted after the deadline, once every submitted policy has been evaluated on the held-out set on a Google Cloud TPU. Resubmission is allowed until the deadline. No written report is required for the bonus.

Acknowledgment. The TPU evaluation for this bonus is supported by Google Cloud TPU credits from the Google TPU Builders program.

[1] TypeSafe, Introducing System One Models & Jev, September 15, 2026.
[2] Together AI, Jev-style classification tutorial, September 23, 2026.